RBAC — project brief

Internal design summary

Summary

We are building a single, central application (working name RBAC) that owns roles and permissions for all of our products. Instead of every application implementing its own roles, screens and role-to-screen mapping, each application will register its screens with the RBAC service and ask it one question at runtime: what is this user allowed to do?

Scope note: RBAC handles authorization only. Logging a user in stays with our existing login / identity provider. RBAC does not store passwords and does not issue sessions.

The problem we have today

- We run several applications, and each one has its own role-based access control built into it.
- In every app the pattern is identical: create a role, map screens to the role, attach the role to a user. The same code and the same tables, written again and again.
- Because each app owns its own copy, there is no single place to see what a person can access across the business, and no shared audit trail of who granted what.
- Every new application means rebuilding the same feature from scratch.
- Changes to the pattern (a new permission type, a fix, an audit requirement) have to be applied in each app separately, and in practice they drift apart.

The solution

One RBAC service plus an admin console. The split of ownership is the important idea:

Who owns it	What they own
Each application	Its own catalogue of screens and the actions available on them
RBAC service	Roles, permission grants, user assignments, audit history

An application knows its own screens, so it declares them. RBAC does not try to guess them and an admin does not type them in by hand — that is how the list drifts out of sync with the code.

Decisions already made

1. **Authorization only.** Identity comes from the existing IdP. RBAC stores a mirror of the user keyed by external_id, not the credentials.
2. **Screen-level permissions with actions.** A permission is a screen plus an action (read, write, update, delete, export). Actions are data, not a fixed enum — each application defines its own action list, so a project can add approve or publish later without a schema change. No record or row-level permissions.
3. **No scoping.** One set of roles per application per user. A user may hold more than one role in an app; their effective permissions are the union of those roles.
4. **RBAC manages itself.** The admin console is registered as just another application (rbac-console) with its own screens and its own roles, stored in the same tables.
5. **Applications cache, they do not call per screen.** Permissions are fetched once per session and cached, not looked up on every render.

Data model

Nine tables. Names are indicative, not final.

- `application` — one row per product. Holds the client credentials it uses to call us.
- `action` — the action catalogue, scoped to an application (read, export, …).
- `resource` — a screen. Has parent_id so screens form a tree (this drives navigation menus).
- `permission` — thin join of resource + action, unique on the pair.
- `role` — scoped to an application. Roles are never shared across applications.
- `role_permission` — which permissions a role grants.
- `app_user` — mirror of the IdP identity (external_id, email).
- `user_role` — assignments, unique on (user_id, role_id).
- `audit_log` — every change: actor, entity, before, after, timestamp.

Permissions get a canonical string form (`invoices.list:export`) so application code and the SDK never handle UUIDs.

Integration approach

There are two separate phases. Do not merge them.

Build time — manifest sync

On deploy, an application posts its permission manifest to RBAC. The endpoint is idempotent.

```
{
  "app": "billing",
  "version": "2026.08.11",
  "actions": ["read", "write", "update", "delete", "export"],
  "resources": [
    { "key": "invoices", "name": "Invoices",
      "actions": ["read", "write", "update", "delete", "export"] },
    { "key": "invoices.credit_note", "name": "Credit notes",
      "parent": "invoices", "actions": ["read", "write"] }
  ]
}
```

Critical rule: the sync must never hard-delete a permission that a role already references. Mark removed ones deprecated and surface them in the console as orphaned. A hard delete means a bad deploy silently strips people's access.

Runtime — one call per session

```
GET /v1/me/permissions?app=billing

{
  "version": 4471,
  "roles": ["billing_clerk"],
  "permissions": ["invoices:read", "invoices:write", "invoices:export"],
  "menu": [ { "key": "invoices", "name": "Invoices", "children": [] } ]
}
```

- `permissions` is what the app checks against, via an SDK can() helper and route middleware.
- `menu` is the screen tree the user may see, so each app renders navigation from RBAC instead of reimplementing "which screens does this role get".
- `version` is the cache key. It is bumped whenever a role or an assignment changes; the SDK refetches on mismatch instead of polling.

Server-side enforcement still applies. The cached snapshot decides what the UI shows; every protected endpoint must check the permission itself.

Things to watch

- **Bootstrap.** Permissions cannot be granted before they exist. The first migration must seed the rbac-console application, its screens, actions and permissions, a SUPER_ADMIN role wired to all of them, and one initial user assignment.
- **Delegated admin is record-level.** If we later want App A's owner to manage only App A's roles, screen-level permissions cannot express that. The cheap fix is a narrow user_application table checked only by console endpoints — not a general policy engine. Not needed for v1.
- **Migration from the existing apps.** Each app currently has live roles and assignments that need importing. Plan for a one-off import per app and a period where both paths work.

-